

OEM Interest Rate Sheet Automation

Multi-format data extraction and change detection — Python & Excel

THE PROBLEM

Vehicle manufacturers' finance arms (OEMs) each send updated consumer lending rates on a regular cycle, broken out by loan term (24-72 months). Every OEM formats their sheet differently. Some flag exactly which rates changed; most don't. Some deliver Excel, some deliver PDF. Comparing each sheet against the last cycle by hand was slow and easy to get wrong.

THE APPROACH

Started with an Excel VBA version to prove the concept. Once the manager asked for something faster, rebuilt it in Python: a dedicated parser per OEM format, all normalizing to one common structure, with automated change-detection against the prior cycle. Runtime dropped from a manual, multi-hour review to under two minutes.

WHAT IT OUTPUTS

A single clean workbook per cycle, with four sheets:

- Clean_Data — standardized rates, ready for entry into the system
- Changes — rates that moved since last cycle, marked in red
- Added_Vehicles — new models introduced this cycle
- Removed_Vehicles — models dropped from an OEM's sheet

THIS DEMO

Rebuilt with four fictional OEMs and fabricated sample data, matching the real format variety: a simple table, a model-by-term matrix, a messy layout with merged header rows, and a PDF-sourced sheet.

OEM	Model	Term_Months	Rate_Percent
Atlas Auto Finance	Horizon Limited	24	4.38
Atlas Auto Finance	Horizon Limited	36	4.78
Atlas Auto Finance	Horizon Limited	48	5.38
Atlas Auto Finance	Horizon Limited	60	5.8
Atlas Auto Finance	Horizon Limited	72	6.11
Atlas Auto Finance	Pinnacle GT	24	4.53
Atlas Auto Finance	Pinnacle GT	36	4.9
Atlas Auto Finance	Pinnacle GT	48	5.29
Atlas Auto Finance	Pinnacle GT	60	5.78
Atlas Auto Finance	Pinnacle GT	72	6.24
Atlas Auto Finance	Pinnacle SE	24	4.62
Atlas Auto Finance	Pinnacle SE	36	4.78
Atlas Auto Finance	Pinnacle SE	48	5.38
Atlas Auto Finance	Pinnacle SE	60	5.64
Atlas Auto Finance	Pinnacle SE	72	6.21
Atlas Auto Finance	Pinnacle Touring	24	4.43
Atlas Auto Finance	Pinnacle Touring	36	4.8
Atlas Auto Finance	Pinnacle Touring	48	5.3
Atlas Auto Finance	Pinnacle Touring	60	5.61
Atlas Auto Finance	Pinnacle Touring	72	6.06
Atlas Auto Finance	Summit XR	24	4.41
Atlas Auto Finance	Summit XR	36	4.85
Atlas Auto Finance	Summit XR	48	5.4
Atlas Auto Finance	Summit XR	60	5.81
Atlas Auto Finance	Summit XR	72	6.52
Crestline Financial	Odyssey Base	24	4.31
Crestline Financial	Odyssey Base	36	4.81
Crestline Financial	Odyssey Base	48	5.4
Crestline Financial	Odyssey Base	60	5.61
Crestline Financial	Odyssey Base	72	6.17
Crestline Financial	Odyssey Plus	24	4.4
Crestline Financial	Odyssey Plus	36	4.96
Crestline Financial	Odyssey Plus	48	5.39
Crestline Financial	Odyssey Plus	60	5.85
Crestline Financial	Odyssey Plus	72	6.12
Crestline Financial	Vantage Elite	24	4.36
Crestline Financial	Vantage Elite	36	4.81
Crestline Financial	Vantage Elite	48	5.17
Crestline Financial	Vantage Elite	60	5.78

OEM	Model	Term_Months	Previous Rate (%)	New Rate (%)
Atlas Auto Finance	Pinnacle SE	24	4.28	4.62
Atlas Auto Finance	Summit XR	72	6.13	6.52
Crestline Financial	Vantage RS	60	5.82	6.17
Meridian Motors	Solstice LX	36	4.74	5.09
Meridian Motors	Trailhawk Base	60	5.76	6.13
Meridian Motors	Trailhawk XT	48	5.27	5.64
Northbridge Capital	Cascade Premium	36	4.7	5.1

OEM	Model
Atlas Auto Finance	Pinnacle GT
Crestline Financial	Vantage Elite
Meridian Motors	Solstice GT
Northbridge Capital	Cascade RS

OEM	Model
Atlas Auto Finance	Horizon Base
Crestline Financial	Frontier LX
Meridian Motors	Voyager Premium
Northbridge Capital	Alpine EX

```
1  """
2  OEM Interest Rate Sheet Automation
3  -----
4  Every finance cycle, each OEM (vehicle manufacturer's captive finance arm)
5  sends over their current consumer lending rates by term (24/36/48/60/72
6  months). These sheets need to end up in the internal system as a clean,
7  standardized table before the data entry team can act on them.
8
9  The problem this tool solves: every OEM formats their sheet differently.
10 Some flag what changed, some don't. Some deliver Excel, some deliver PDF.
11 Some list rates in a simple table, some in a matrix, some bury the table
12 under header notes with merged cells. None of that is consistent, and it
13 used to mean manually opening each file and eyeballing it against last
14 cycle's numbers.
15
16 This tool:
17     1. Reads every OEM's rate sheet from INPUT_DIR, using a dedicated
18        parser per OEM format (because no two formats are alike).
19     2. Normalizes everything to one common structure: OEM, Model,
20        Term (Months), Rate (%).
21     3. Compares the result against last cycle's saved snapshot
22        (REFERENCE_DIR) to detect what changed, what's new, and what
23        was dropped.
24     4. Writes one clean Excel workbook to OUTPUT_DIR with four sheets:
25        - Clean_Data      : the full, standardized rate table, ready
26                           for the data entry team
27        - Changes         : rows where the rate moved since last
28                           cycle, old vs. new value, highlighted red
29        - Added_Vehicles  : models that are brand new this cycle
30        - Removed_Vehicles : models that disappeared from an OEM's
31                           sheet since last cycle
32     5. Saves the new Clean_Data as this cycle's reference snapshot,
33        so next cycle's comparison has something to diff against.
34
35 Adding a new OEM later just means writing one new parser function and
36 registering it in OEM_PARSERS below – nothing else in the pipeline changes.
37
38 Author: Majd Kabbani
39 """
40
41 import os
42 from datetime import date
```

```
43
44 import pandas as pd
45 import pdfplumber
46 import openpyxl
47 from openpyxl.styles import Font, PatternFill, Alignment
48 from openpyxl.utils import get_column_letter
49
50 INPUT_DIR = "input"
51 REFERENCE_DIR = "reference"
52 OUTPUT_DIR = "output"
53
54 REFERENCE_FILE = os.path.join(REFERENCE_DIR, "previous_clean_data.xlsx")
55 OUTPUT_FILE = os.path.join(OUTPUT_DIR, f"clean_rates_{date.today().isoformat()}.xlsx")
56
57 COLUMNS = ["OEM", "Model", "Term_Months", "Rate_Percent"]
58
59
60 # -----
61 # OEM-SPECIFIC PARSERS
62 # Each OEM formats their sheet differently, so each gets its own parser.
63 # Every parser's only job is to return a clean DataFrame with columns:
64 # OEM, Model, Term_Months, Rate_Percent
65 # -----
66
67 def parse_meridian(path: str) -> pd.DataFrame:
68     """Simple table format. Has its own 'Change Flag' column, but we still
69     compute changes independently against the reference so every OEM is
70     treated consistently regardless of whether they self-report changes."""
71     df = pd.read_excel(path, header=3) # first 3 rows are title/date/blank
72     df = df.rename(columns={"Term (Months)": "Term_Months", "Rate (%)": "Rate_Percent"})
73     df["OEM"] = "Meridian Motors"
74     return df[COLUMNS]
75
76
77 def parse_atlas(path: str) -> pd.DataFrame:
78     """Matrix format: one row per model, one column per term. No change
79     flag at all – needs to be melted into long format."""
80     df = pd.read_excel(path)
81     term_cols = [c for c in df.columns if c != "Model"]
82     long_df = df.melt(id_vars="Model", value_vars=term_cols,
83                      var_name="Term_Months", value_name="Rate_Percent")
84     long_df["Term_Months"] = long_df["Term_Months"].str.replace(" mo", "", regex=False).astype(int)
```

```
85     long_df["OEM"] = "Atlas Auto Finance"
86     return long_df[COLUMNS]
87
88
89 def parse_northbridge(path: str) -> pd.DataFrame:
90     """Messy layout: 3 header/notice rows before the real table starts,
91     and the model name is only present on the first term row per model
92     (blank on the rest, since it's visually 'grouped' in the source file).
93     Needs a forward-fill to reconstruct the model for every row."""
94     df = pd.read_excel(path, header=None, skiprows=4)
95     df.columns = ["Model", "Term_Months", "Rate_Percent"]
96     df["Model"] = df["Model"].ffill()
97     df["OEM"] = "Northbridge Capital"
98     return df[COLUMNS]
99
100
101 def parse_crestline_pdf(path: str) -> pd.DataFrame:
102     """PDF source. Table is extracted page by page with pdfplumber rather
103     than read directly like the Excel-based OEMs."""
104     rows = []
105     with pdfplumber.open(path) as pdf:
106         for page in pdf.pages:
107             table = page.extract_table()
108             if not table:
109                 continue
110             headers, *data_rows = table
111             for row in data_rows:
112                 rows.append(dict(zip(headers, row)))
113     df = pd.DataFrame(rows)
114     df = df.rename(columns={"Term (Months)": "Term_Months", "Rate (%)": "Rate_Percent"})
115     df["Term_Months"] = df["Term_Months"].astype(int)
116     df["Rate_Percent"] = df["Rate_Percent"].astype(float)
117     df["OEM"] = "Crestline Financial"
118     return df[COLUMNS]
119
120
121 # Registry: filename pattern -> parser function.
122 # Adding a new OEM later means adding one line here.
123 OEM_PARSERS = {
124     "meridian_current.xlsx": parse_meridian,
125     "atlas_current.xlsx": parse_atlas,
126     "northbridge_current.xlsx": parse_northbridge,
```



```
127     "crestline_current.pdf": parse_crestline_pdf,
128 }
129
130
131 # -----
132 # PIPELINE
133 # -----
134 def load_all_current_sheets() -> pd.DataFrame:
135     frames = []
136     for filename, parser in OEM_PARSERS.items():
137         path = os.path.join(INPUT_DIR, filename)
138         if not os.path.exists(path):
139             print(f" [skip] {filename} not found in input folder")
140             continue
141         df = parser(path)
142         frames.append(df)
143         print(f" [ok] {filename}: {len(df)} rate rows parsed")
144     return pd.concat(frames, ignore_index=True)
145
146
147 def compare_to_reference(current: pd.DataFrame, previous: pd.DataFrame) -> dict:
148     key_cols = ["OEM", "Model", "Term_Months"]
149     merged = current.merge(previous, on=key_cols, how="outer",
150                           suffixes=("_current", "_previous"), indicator=True)
151
152     # Changed: present in both, but rate differs
153     both = merged[merged["_merge"] == "both"].copy()
154     changed = both[both["Rate_Percent_current"] != both["Rate_Percent_previous"]].copy()
155     changed = changed.rename(columns={
156         "Rate_Percent_previous": "Previous Rate (%)",
157         "Rate_Percent_current": "New Rate (%)",
158     })[["OEM", "Model", "Term_Months", "Previous Rate (%)", "New Rate (%)"]]
159
160     # Added vehicles: models present in current for an OEM, absent in previous for that OEM
161     current_models = set(current[["OEM", "Model"]].itertuples(index=False, name=None))
162     previous_models = set(previous[["OEM", "Model"]].itertuples(index=False, name=None))
163     added_models = current_models - previous_models
164     removed_models = previous_models - current_models
165
166     added_df = pd.DataFrame(sorted(added_models), columns=["OEM", "Model"])
167     removed_df = pd.DataFrame(sorted(removed_models), columns=["OEM", "Model"])
168
```

```
169     return {"changed": changed, "added": added_df, "removed": removed_df}
170
171
172 # -----
173 # OUTPUT WORKBOOK
174 # -----
175 def style_header(ws, ncols, row=1):
176     fill = PatternFill("solid", fgColor="1F3864")
177     font = Font(bold=True, color="FFFFFF")
178     for c in range(1, ncols + 1):
179         cell = ws.cell(row=row, column=c)
180         cell.fill = fill
181         cell.font = font
182         cell.alignment = Alignment(horizontal="center")
183
184
185 def autosize(ws, ncols, width=18):
186     for c in range(1, ncols + 1):
187         ws.column_dimensions[get_column_letter(c)].width = width
188
189
190 def write_df(ws, df: pd.DataFrame, highlight_col=None):
191     headers = list(df.columns)
192     for j, h in enumerate(headers, start=1):
193         ws.cell(row=1, column=j, value=h)
194     style_header(ws, len(headers))
195     red_fill = PatternFill("solid", fgColor="FDE7E7")
196     red_font = Font(color="B5493A", bold=True)
197     for i, (_, row) in enumerate(df.iterrows(), start=2):
198         for j, h in enumerate(headers, start=1):
199             cell = ws.cell(row=i, column=j, value=row[h])
200             if highlight_col and h == highlight_col:
201                 cell.fill = red_fill
202                 cell.font = red_font
203     autosize(ws, len(headers))
204
205
206 def build_output_workbook(clean_df, diffs):
207     wb = openpyxl.Workbook()
208
209     ws = wb.active
210     ws.title = "Clean_Data"
```

```
211     write_df(ws, clean_df.sort_values(["OEM", "Model", "Term_Months"]))
212
213     ws = wb.create_sheet("Changes")
214     if not diffs["changed"].empty:
215         write_df(ws, diffs["changed"], highlight_col="New Rate (%)")
216     else:
217         ws["A1"] = "No rate changes this cycle."
218
219     ws = wb.create_sheet("Added_Vehicles")
220     if not diffs["added"].empty:
221         write_df(ws, diffs["added"])
222     else:
223         ws["A1"] = "No new vehicles this cycle."
224
225     ws = wb.create_sheet("Removed_Vehicles")
226     if not diffs["removed"].empty:
227         write_df(ws, diffs["removed"])
228     else:
229         ws["A1"] = "No vehicles removed this cycle."
230
231     os.makedirs(OUTPUT_DIR, exist_ok=True)
232     wb.save(OUTPUT_FILE)
233
234
235 # -----
236 # MAIN
237 # -----
238 def main():
239     print("Parsing current-cycle OEM rate sheets...")
240     current = load_all_current_sheets()
241
242     previous = pd.read_excel(REFERENCE_FILE)
243
244     print("Comparing against last cycle's reference snapshot...")
245     diffs = compare_to_reference(current, previous)
246
247     print("Writing clean output workbook...")
248     build_output_workbook(current, diffs)
249
250     # This cycle's clean data becomes next cycle's reference
251     current.to_excel(REFERENCE_FILE, index=False)
252
```

```
253     print(f"\nDone. Report saved to: {OUTPUT_FILE}")
254     print(f"   Total rate rows: {len(current)}")
255     print(f"   Changed: {len(diffs['changed'])}")
256     print(f"   Added vehicles: {len(diffs['added'])}")
257     print(f"   Removed vehicles: {len(diffs['removed'])}")
258
259
260 if __name__ == "__main__":
261     main()
262
```